

P6: Working with Multiples

Name: Alston Liow Guan You

Student ID: 37440985

Date: 16 August 2026

Overview

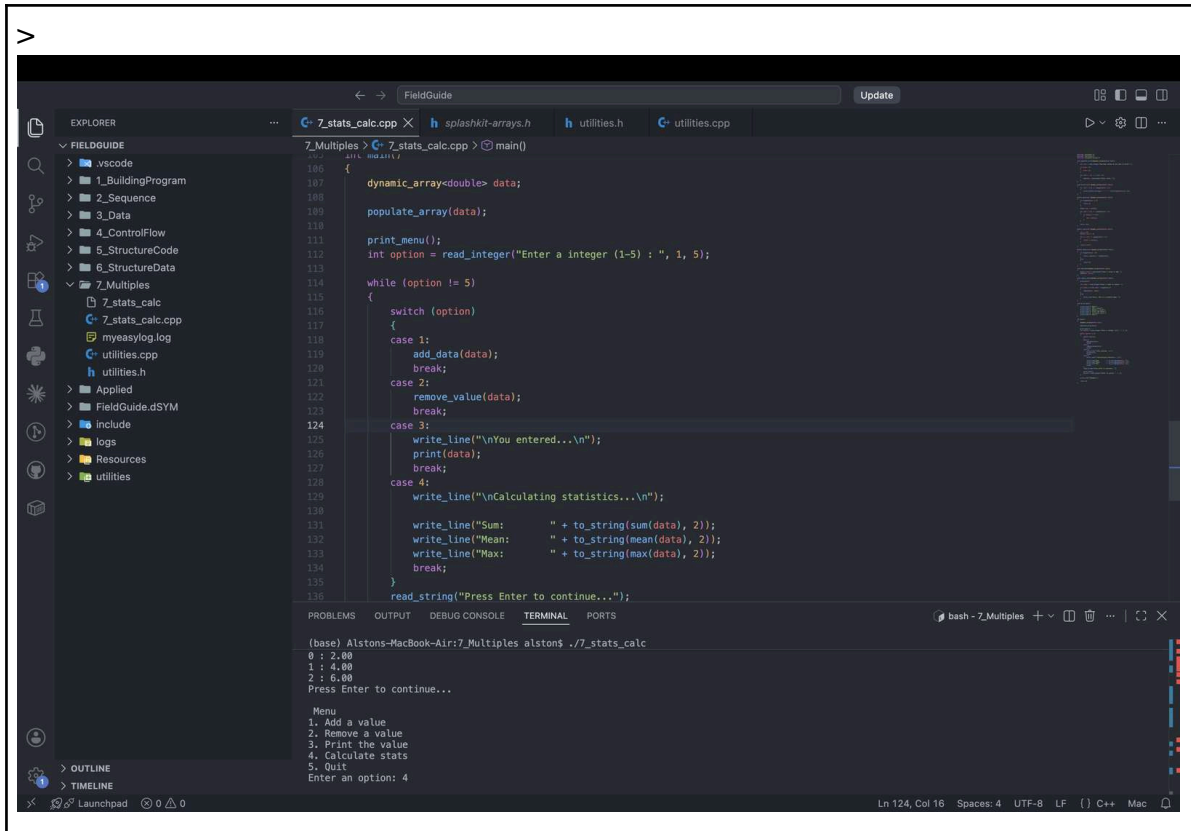
This task is an opportunity to explore and use language mechanisms to allow you to easily work with multiple values, which can grow and shrink in number as the program executes.

Tip: Systems thinking

Work to develop your ability to see how this comes together as a system. The parts are not independent of each other. Understanding each part in isolation is a good start. Make sure to follow this up by seriously focusing on seeing how the parts come together and work as a system. When you can see this, you can start to think about how to build and manipulate these systems, getting them to achieve what you want.

Do the following:

- Everything you need is in the Handling Multiples chapter of our [Programmer's Field Guide](#). The [Intro Tours](#) walk through the concepts, and should give you a good start.
- Add a screenshot image showing you were able to create and work with arrays of simple values - using a dynamic array, using interaction, for performing actions like searching, expanding, removing, and accumulating values. You can use the [statistics calculator tour](#) to achieve this (multiple pages).



The screenshot shows a C++ IDE with a project named 'FieldGuide'. The file explorer on the left shows a directory structure with files like '7_stats_calc.cpp', 'utilities.h', and 'utilities.cpp'. The main editor displays the code for '7_stats_calc.cpp', which includes a 'main()' function. The code uses a 'dynamic_array<double>' to store data, populates it, and then enters a loop where the user can add, remove, or calculate statistics on the data. The terminal at the bottom shows the program's output, including a menu and the results of calculations.

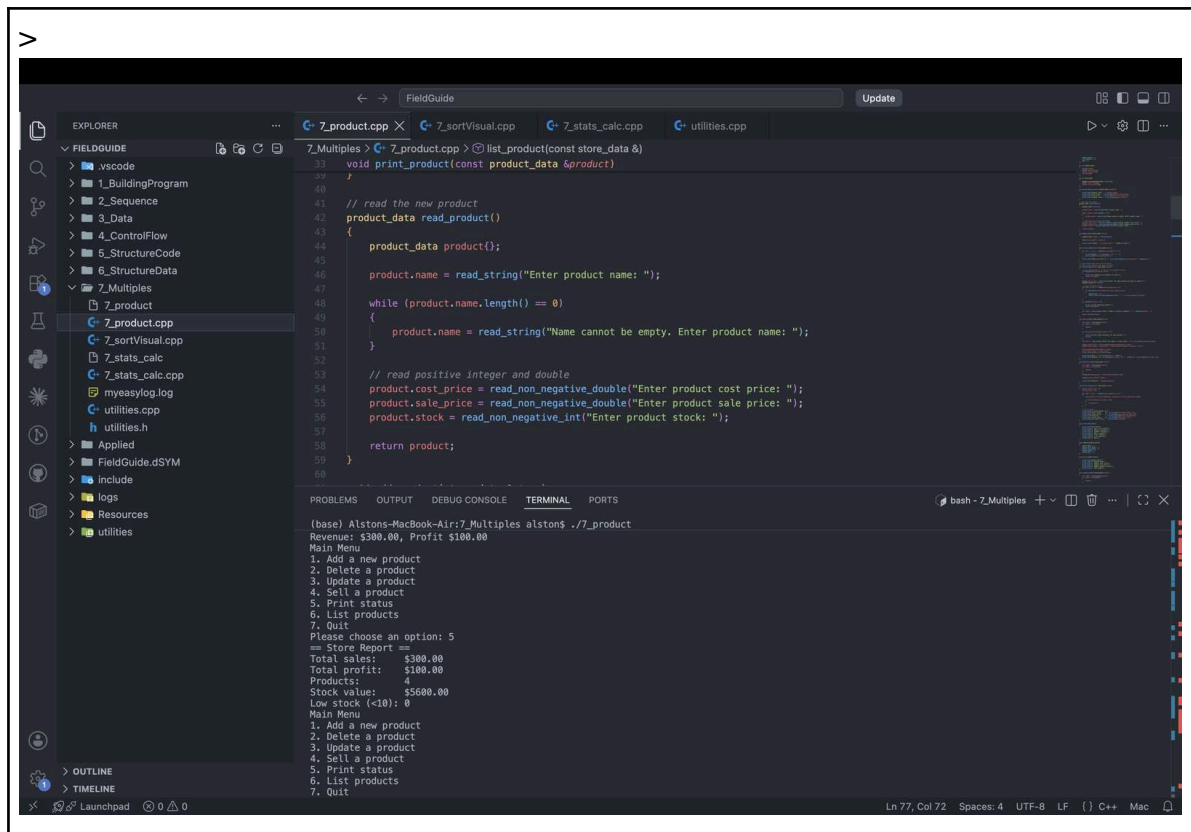
```
106 {
107     dynamic_array<double> data;
108     populate_array(data);
109
110     print_menu();
111     int option = read_integer("Enter a Integer (1-5) : ", 1, 5);
112
113     while (option != 5)
114     {
115         switch (option)
116         {
117             case 1:
118                 add_data(data);
119                 break;
120             case 2:
121                 remove_value(data);
122                 break;
123             case 3:
124                 write_line("\nYou entered...\n");
125                 print(data);
126                 break;
127             case 4:
128                 write_line("\nCalculating statistics...\n");
129
130                 write_line("Sum:      " + to_string(sum(data), 2));
131                 write_line("Mean:    " + to_string(mean(data), 2));
132                 write_line("Max:     " + to_string(max(data), 2));
133                 break;
134         }
135         read_string("Press Enter to continue...");
136     }
137 }
```

Terminal Output:

```
(base) Alstons-MacBook-Air:7_Multiples alston$ ./7_stats_calc
0 : 2.00
1 : 4.00
2 : 6.00
Press Enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculate stats
5. Quit
Enter an option: 4
```

- Add a screenshot image showing you were able to create and use arrays of your own custom types (structs). You can use the [product database tour](#) to achieve this (multiple pages).



- Add a photo showing you can use our [hand execution process](#) to illustrate how code operates on arrays using the following source code.

```
const double VALUES[4] = { 8, 2, 9, 7 };
const int SIZE = 4;

double result = 0;

// Check the rest of the values
for (int i = 0; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```

>

P6 ①

Notes _____ Date: _____

Memory

	0	1	2	3
VALUES	8	2	9	7

result ~~0~~ ~~8~~ 9

i ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4

- Consider the code as a system - what is it aiming to achieve? It does aim to achieve something meaningful - what is that? Can you see any issues with the way this is working in relation to what it is likely aiming to achieve?

> It is aiming to achieve to find the largest number in the arrays which is positive number.

- Repeat the hand execution for the following source code.

```
const double VALUES[4] = { -7, -1, -9, -4 };
const int SIZE = 4;
```

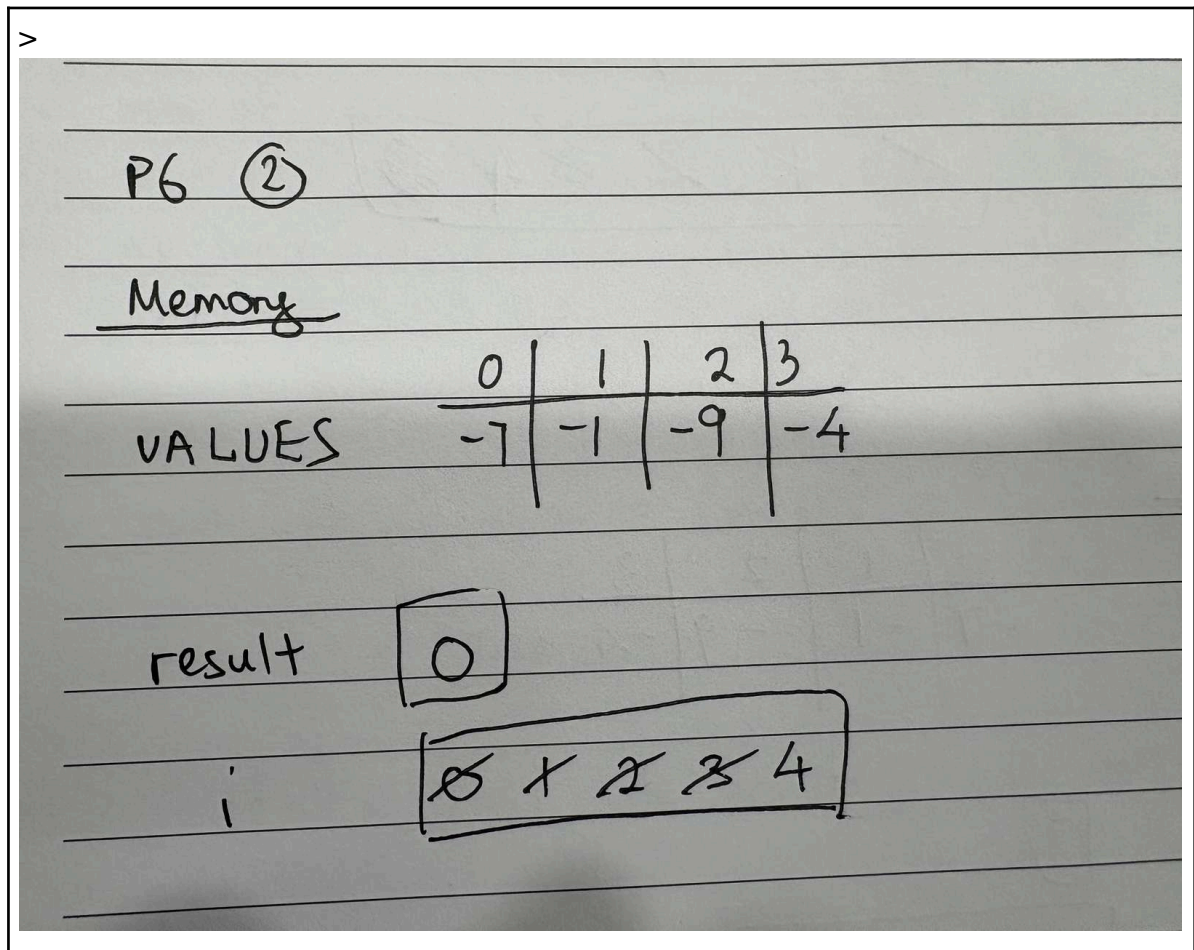
```
double result = 0;
```

```
// Check the rest of the values
for (int i = 0; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```

```

    }
}

```



- Repeat the hand execution for the following source code - which made two small changes to the logic!

```

const double VALUES[4] = { -7, -1, -9, -4 };
const int SIZE = 4;

```

```

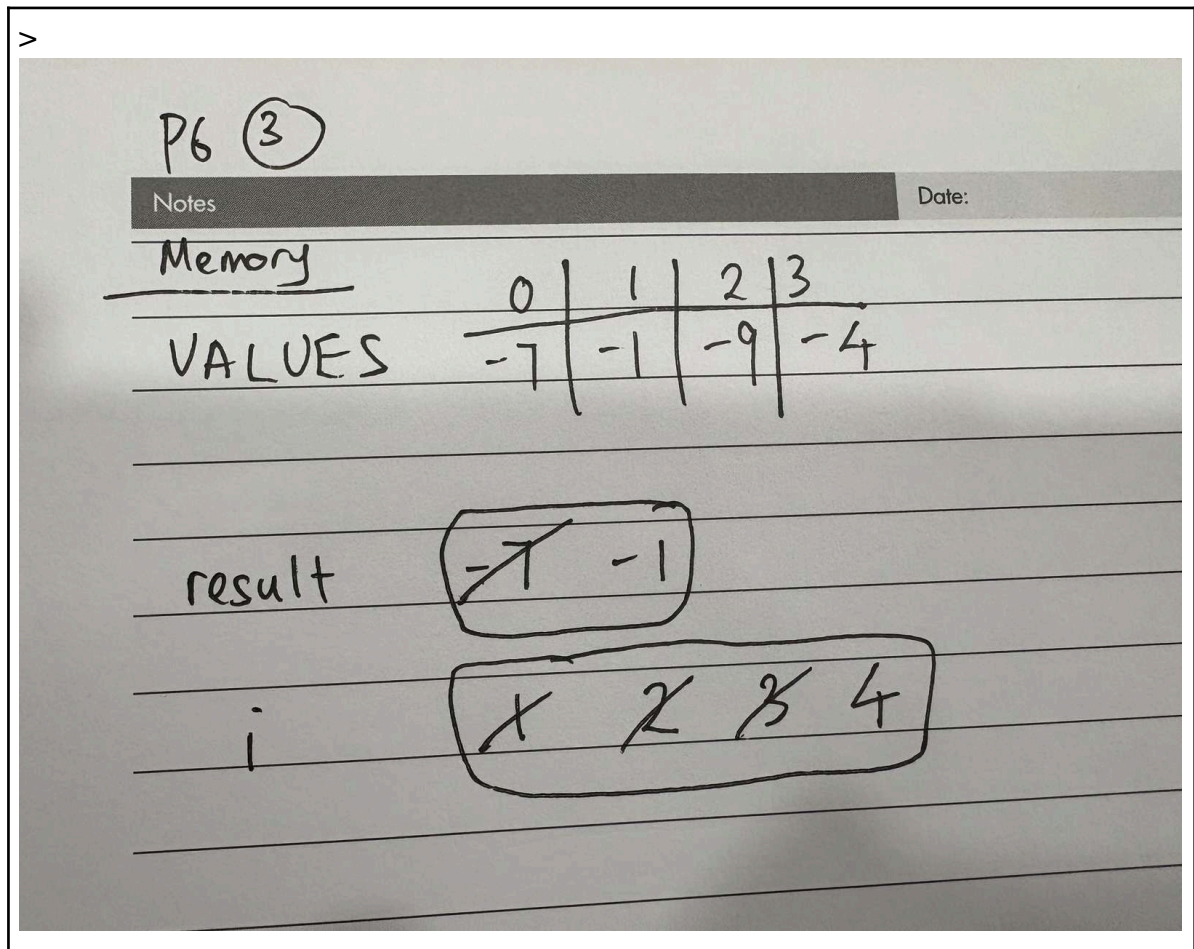
double result = VALUES[0];

```

```

// Check the rest of the values
for (int i = 1; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}

```



- How did the change impact the result? What does this say about the way these systems are put together? Consider how this impacts cases where others (including AI) are writing the code for you.

> Before changing, the result in the VALUE compare its with 0, after changing, it compare with VALUE[0], which inside the VALUE, so the changing make it can find the largest number inside the Arrays.

Tip: Focus and attention to details

Computing capabilities require you to be able to be meticulous, pay attention to details, and be able to focus on things for a period of time. These hand execution tasks will exercise these attributes. Make sure you can do this in a distraction free environment, and that you force yourself to focus on just this activity. It is a little mindless, you are wanting to show you *know* what the computer does. But you need to be able to step through things at this level to really see what the computer is doing - it is unintelligent, so we are relying on your intelligence to make it do the right thing!

- Now you need to demonstrate you can perform hand execution on more complex code. You should be able to hand-execute code without understanding the algorithm itself - the computer is unintelligent and it can follow the steps.

```
const int MAX_SIZE = 5;

double data[MAX_SIZE] = {5, 3, 2};
int size = 3;

double temp;
for (int i = 0; i < MAX_SIZE - 1; i++)
{
    for (int j = 0; j < size - i - 1; j++)
    {
        if (data[j] > data[j + 1])
        {
            temp = data[j];
            data[j] = data[j + 1];
            data[j + 1] = temp;
        }
    }
}
```

>

P6 (4)

Memory

	0	1	2	3	4
data	5	3	2		
	3	5	5		
	2	2			
		3			

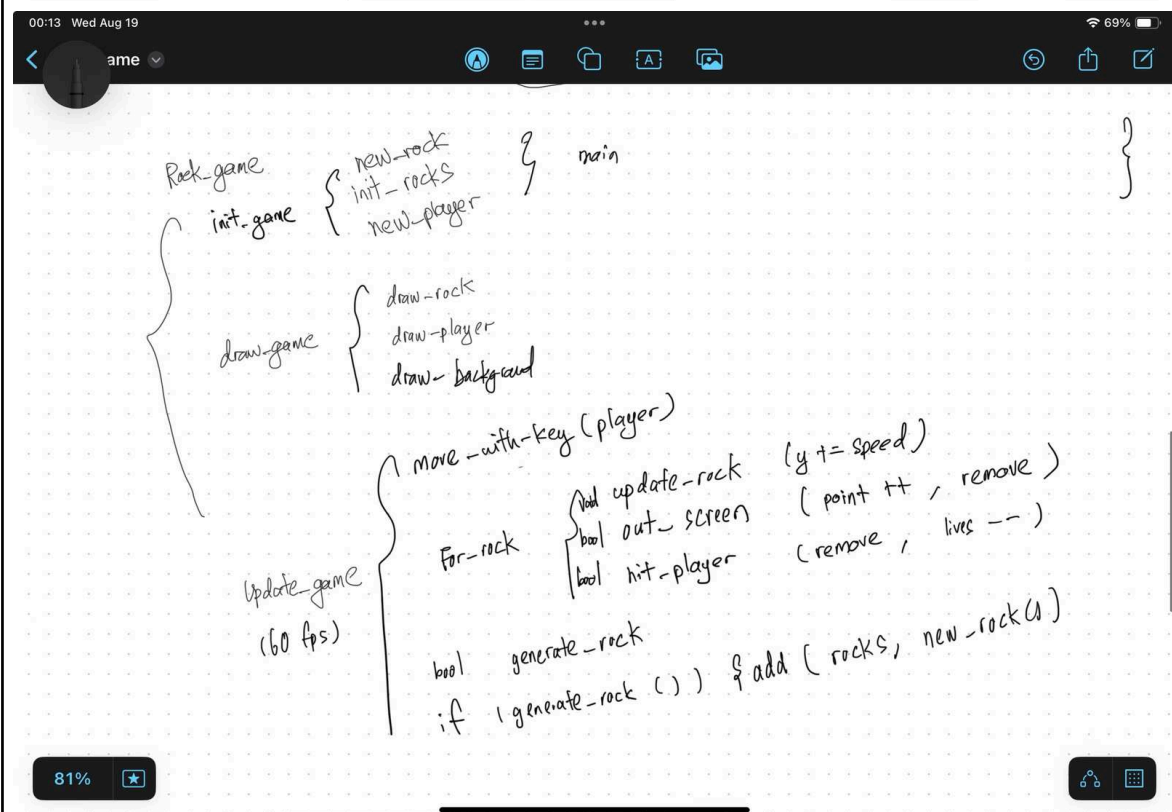
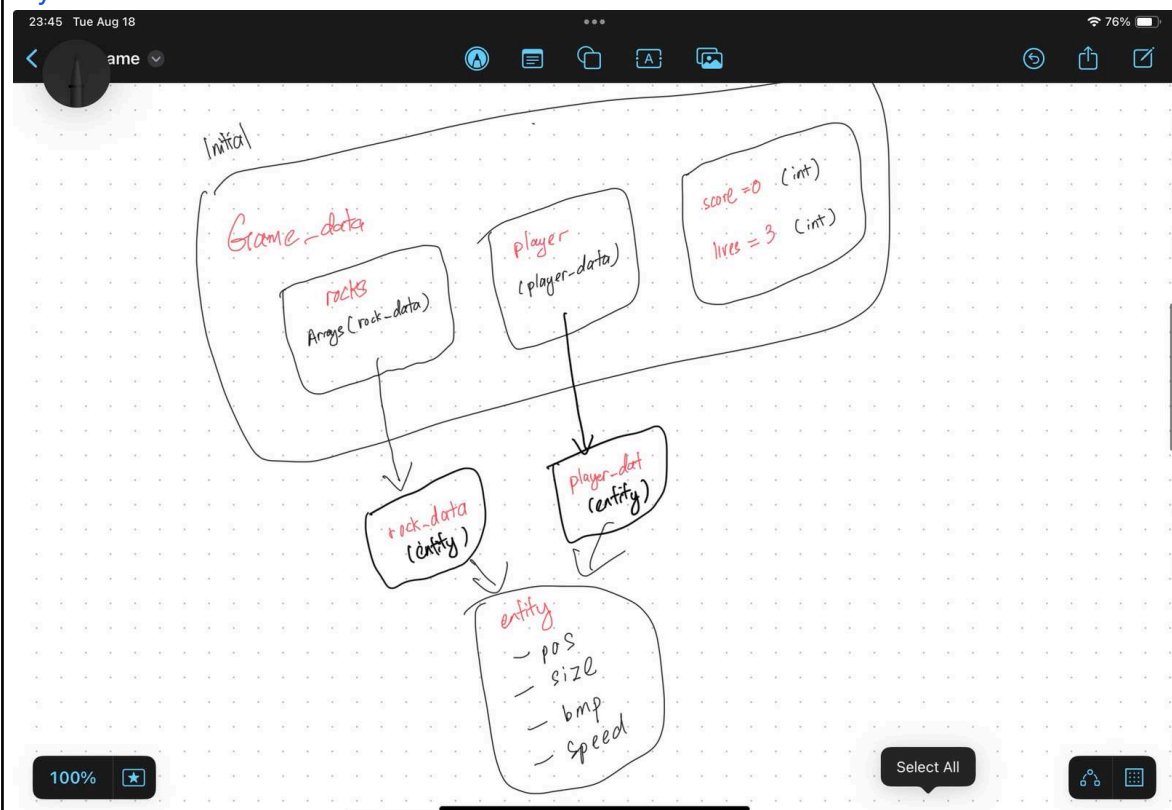
~~test~~

i 0 1 2 3 4

j 0 1 ~~2~~ ~~3~~ 0

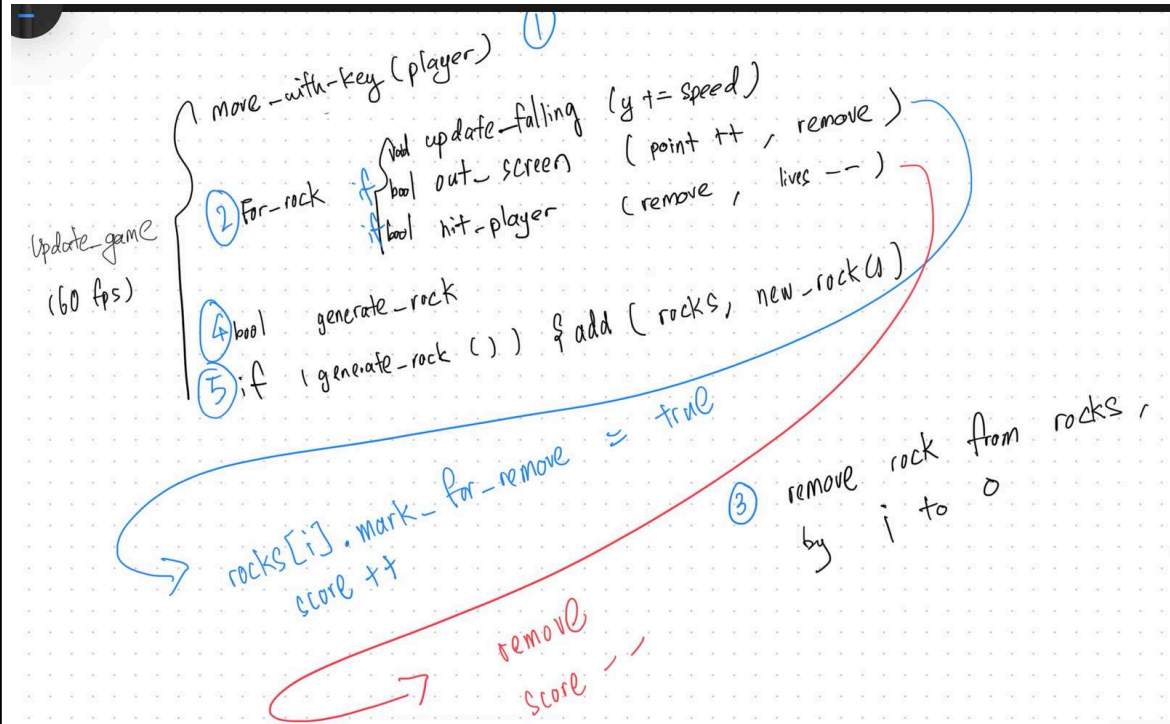
- Complete one of the [Test Your Capabilities](#) tasks from the chapter to see that you can now put these tools together to achieve something yourself.

> I choose the rock game. First I get some reference at the fly catch and map explorer, and plan about the structure of data and function I need to use about this game, this is my initial structure.

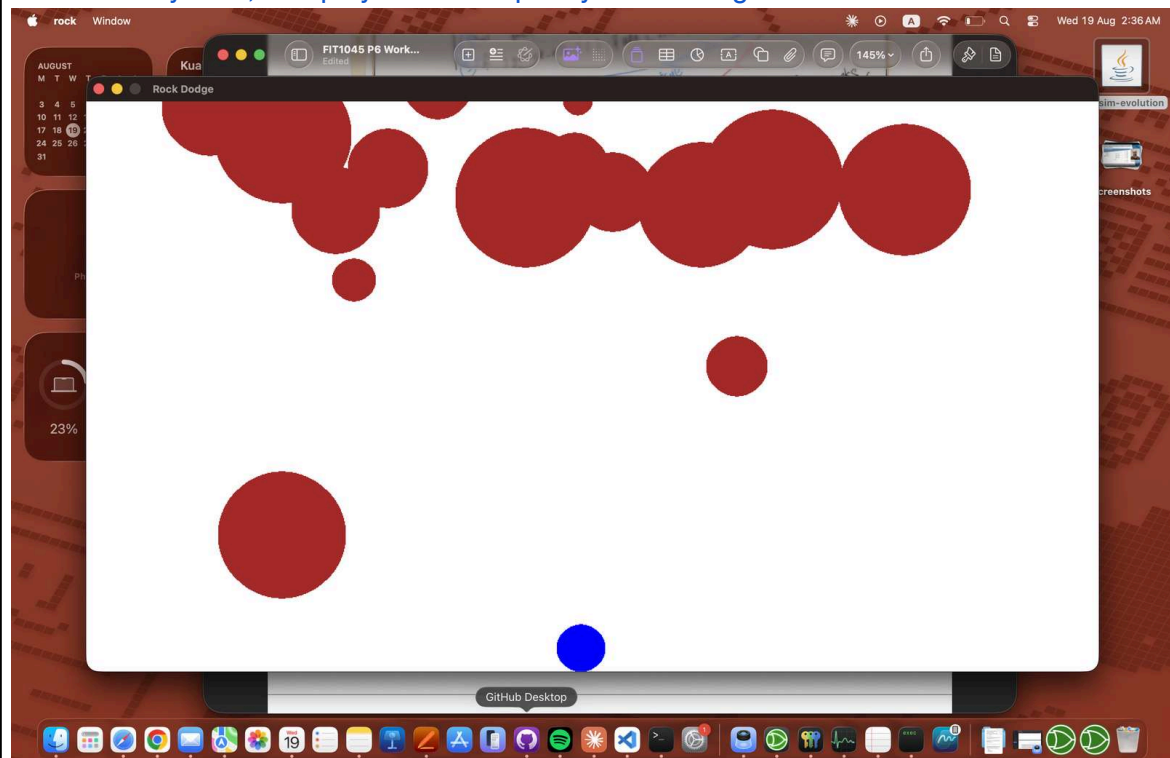


I write all the function step by step until need to combine all update function in to `update_game()`, I get stuck here. I have some idea to add something in this game, for example is typed_down(SPACE_KEY) to jump , and I want to add a horizontal thing like the rock. But I just put beside and do the main thing first.

I analyse the `update_game()` like this



First time I complete this game I forget to add a `const int MAX_ROCK_COUNT = 100`, so it is too many rock, and player will die quickly. Then the game become like this



Now I want to change the circle to using sprite to present the animation. I check the splash kit and try to change my code.

When adding a bitmap, is very hassle to change all the function parameter, so I decide to add bitmap into game_data struct.

The position all get wrong after I change to bitmap and sprite, its very difficult to find the bug, I spend many time and going to have a breakdown.

Finally, I fixed it with some calculation.

FieldGuide

```

7_Multiples > 7_rock > C: rock.cpp > draw_game(game_data &)
You, 3 days ago | 1 author (You)
1 #include "splashkit.h"
2 #include "utilities.h"
3
4
5 const int SCREEN_WIDTH = 1280;
6 const int SCREEN_HEIGHT = 720;
7 const int PLAYER_SPEED = 5;
8
9 const int INIT_ROCK_COUNT = 3;
10 const int MAX_ROCK_COUNT = 100;
11
12 You, 3 days ago | 1 author (You)
13 struct entity
14 {
15     bool enabled(true);
16     point_2d pos();
17     double size();
18     bitmap bmp();
19     int speed();
20     color clr();
21     // sprite spr();
22 };
23
24 // rocks is entity
25 You, 3 days ago | 1 author (You)
26 struct rock_data
27 {
28     entity ent();
29     bool mark_for_remove(false);
30 };
31
32 // player is entity
33 You, 3 days ago | 1 author (You)
34 struct player_data
35 {
36     entity ent();
37 };
38
39 You, 3 days ago | 1 author (You)
40 struct game_data
41 {
42     dynamic_array<rock_data> rocks();
43     player_data player();
44     int score();
45     int lives();
46     bitmap background();
47 };
48
49 // initial the game
50 // contains init_rocks, init_player and new rocks
51
52 // new rock, for one rock
53 // here can setting the difficulty of the game
54 rock_data new_rock(bitmap rock_bmp)

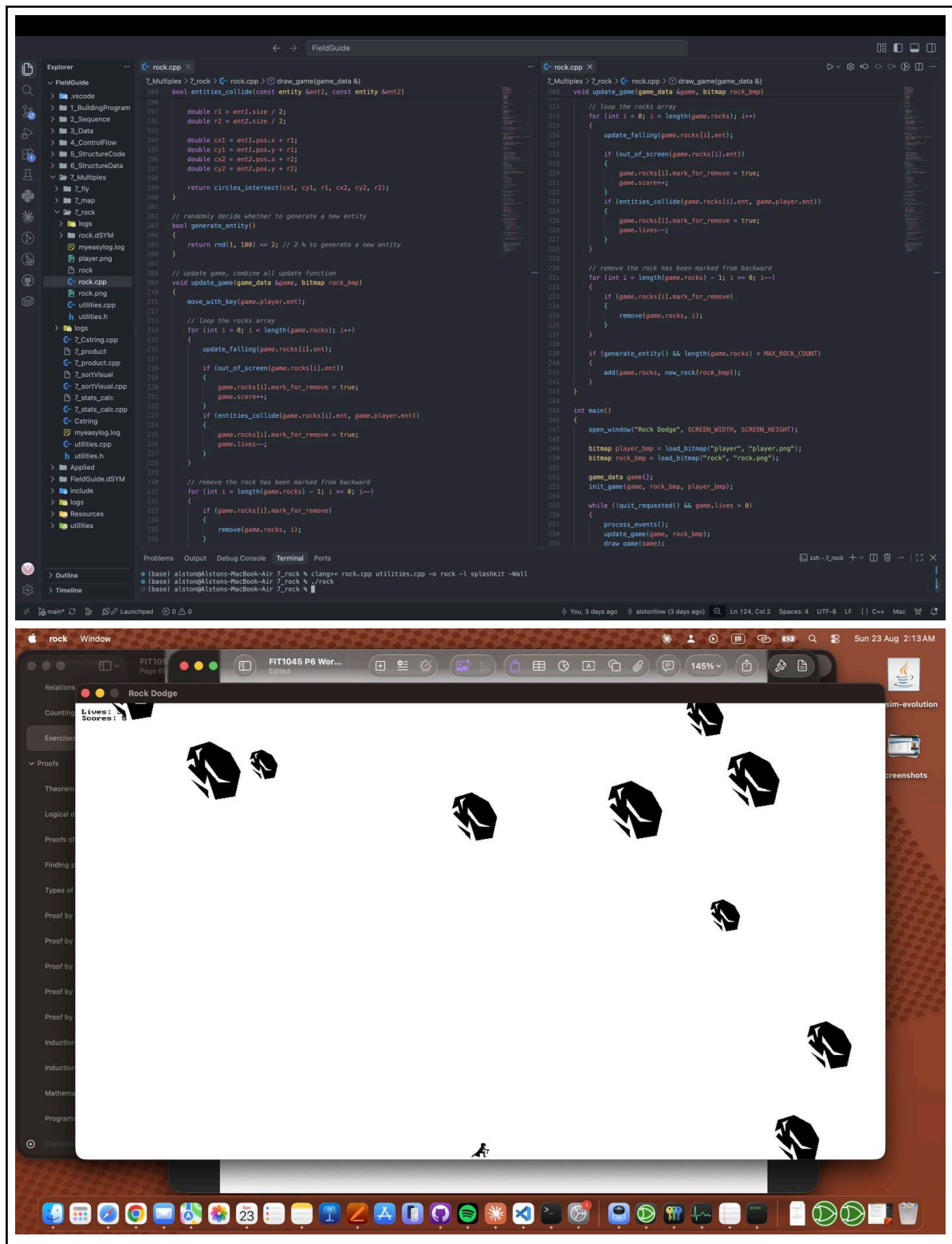
```

FieldGuide

```

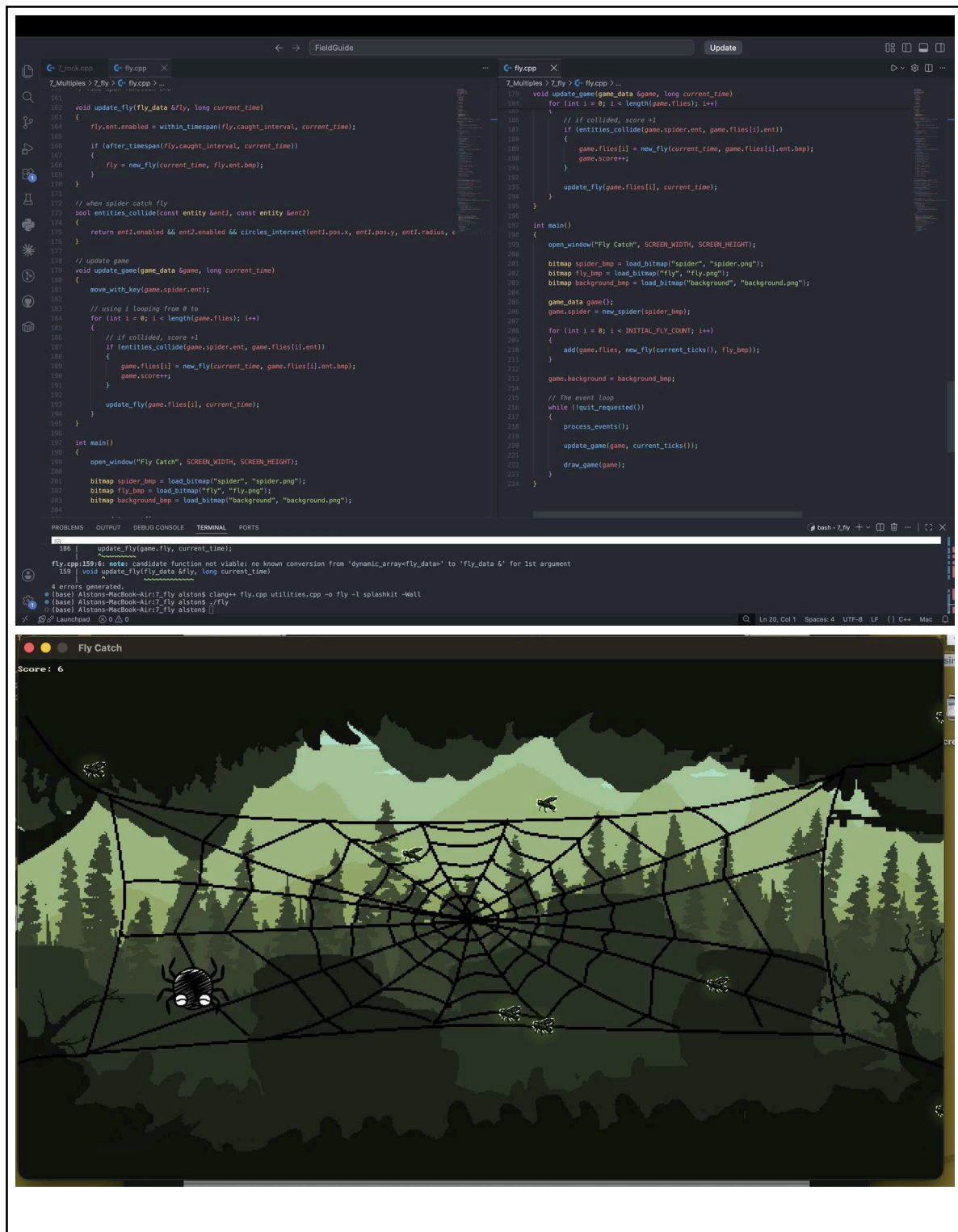
7_Multiples > 7_rock > C: rock.cpp > draw_game(game_data &)
You, 3 days ago | 1 author (You)
55 {
56     rock_data rock();
57
58     rock.ent.size = rnd(30, 100);
59     rock.ent.pos.x = rnd(0, SCREEN_WIDTH - rock.ent.size);
60     rock.ent.pos.y = rnd(0, SCREEN_HEIGHT - rock.ent.size);
61     rock.ent.speed = rnd(1, 5);
62     rock.ent.clr = COLOR_BROWN;
63     rock.ent.bmp = rock_bmp;
64
65     return rock;
66 }
67
68 // init rocks, for many rocks
69 void init_rocks(dynamic_array<rock_data> &rocks, bitmap rock_bmp)
70 {
71     for (int i = 0; i < INIT_ROCK_COUNT; i++)
72     {
73         add(rocks, new_rock(rock_bmp));
74     }
75 }
76
77 // new player data
78 player_data new_player(bitmap player_bmp)
79 {
80     player_data player();
81
82     player.ent.size = 30;
83     player.ent.pos.x = (SCREEN_WIDTH / 2) - (player.ent.size / 2); // player at middle
84     player.ent.pos.y = SCREEN_HEIGHT - player.ent.size; // player at bottom
85     player.ent.speed = PLAYER_SPEED;
86     player.ent.clr = COLOR_BLUE;
87     player.ent.bmp = player_bmp;
88
89     return player;
90 }
91
92 // init game, combine rock and player initial data
93 void init_game(game_data &game, bitmap rock_bmp, bitmap player_bmp)
94 {
95     init_rocks(game.rocks, rock_bmp);
96     game.player = new_player(player_bmp);
97 }
98
99 // draw game
100 void draw_game(game_data &game)
101 {
102     // draw entity
103     void draw_entity(entity &ent)
104     {
105         if (!ent.enabled)
106             return;
107
108         // update game part
109         void update_game_part()
110         {
111             // move with key
112             void move_with_key(entity &ent)
113             {
114                 if (key_down(RIGHT_KEY))
115                 {
116                     ent.pos.x += ent.speed;
117                 }
118                 if (key_down(LEFT_KEY))
119                 {
120                     ent.pos.x -= ent.speed;
121                 }
122                 if ((ent.pos.x + ent.size) > SCREEN_WIDTH)
123                 {
124                     ent.pos.x = SCREEN_WIDTH - ent.size;
125                 }
126                 if (ent.pos.x < 0)
127                 {
128                     ent.pos.x = 0;
129                 }
130             }
131
132             // update the position of rock with its speed with 60fps
133             void update_falling(entity &ent)
134             {
135                 ent.pos.y += ent.speed;
136             }
137
138             // update the position with horizontal
139             void update_hori(entity &ent)
140             {
141                 ent.pos.x += ent.speed;
142             }
143
144             // check if out of screen
145             bool out_of_screen(const entity &ent)
146             {
147                 return ent.pos.y > SCREEN_HEIGHT;
148             }
149
150             // check if when collide
151             bool entities_collide(const entity &ent1, const entity &ent2)
152             {
153                 if (!ent1.enabled || !ent2.enabled)
154                     return false;
155
156                 double r1 = ent1.size / 2;
157                 double r2 = ent2.size / 2;

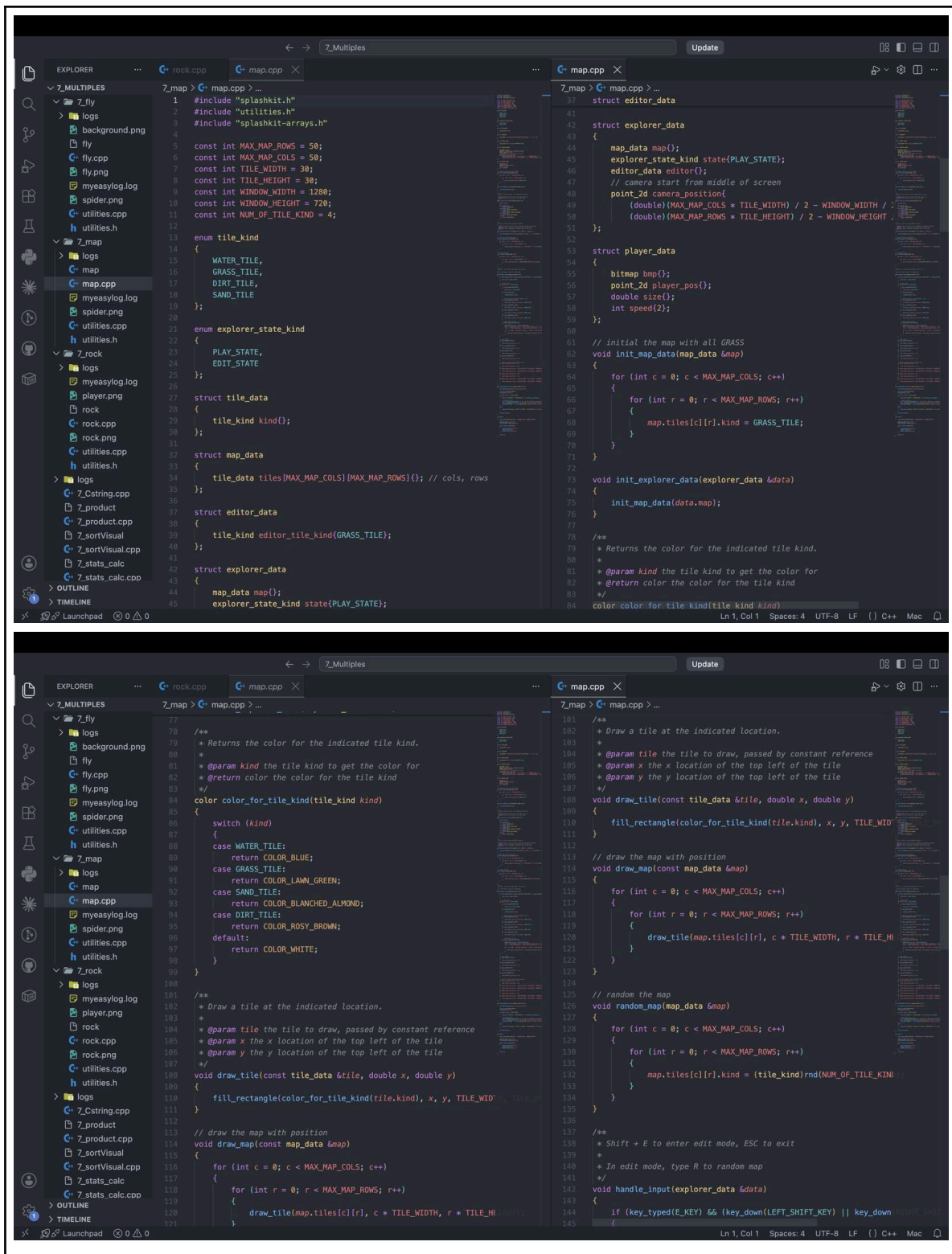
```



- If you want to extend your capabilities further in terms of creating larger programs, add a screenshot here showing what you were able to achieve. We recommend both the [extended fly catch tour](#), and the [map explorer tour](#). Fly catch is a quick addition to the program with structs, while map explorer will get you working with 2-dimensional arrays.





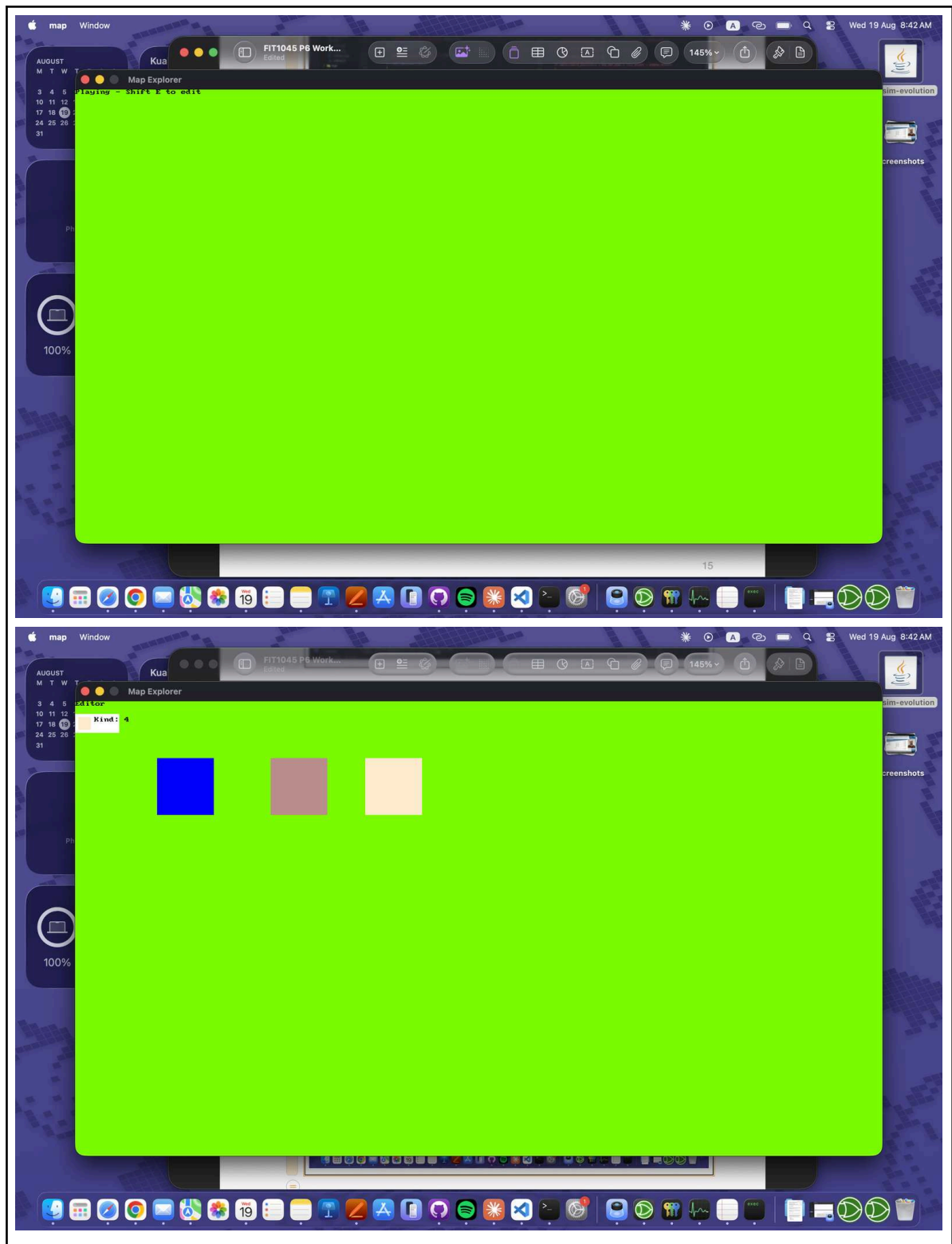


```

1  #include "splashkit.h"
2  #include "utilities.h"
3  #include "splashkit-arrays.h"
4
5  const int MAX_MAP_ROWS = 50;
6  const int MAX_MAP_COLS = 50;
7  const int TILE_WIDTH = 30;
8  const int TILE_HEIGHT = 30;
9  const int WINDOW_WIDTH = 1280;
10 const int WINDOW_HEIGHT = 720;
11 const int NUM_OF_TILE_KIND = 4;
12
13 enum tile_kind
14 {
15     WATER_TILE,
16     GRASS_TILE,
17     DIRT_TILE,
18     SAND_TILE
19 };
20
21 enum explorer_state_kind
22 {
23     PLAY_STATE,
24     EDIT_STATE
25 };
26
27 struct tile_data
28 {
29     tile_kind kind;
30 };
31
32 struct map_data
33 {
34     tile_data tiles[MAX_MAP_COLS][MAX_MAP_ROWS]; // cols, rows
35 };
36
37 struct editor_data
38 {
39     tile_kind editor_tile_kind(GRASS_TILE);
40 };
41
42 struct explorer_data
43 {
44     map_data map;
45     explorer_state_kind state(PLAY_STATE);
46 };
47
48 struct explorer_data
49 {
50     map_data map;
51     explorer_state_kind state(PLAY_STATE);
52 };
53
54 struct player_data
55 {
56     bitmap bmp;
57     point_2d player_pos;
58     double size;
59     int speed(2);
60 };
61
62 // initial the map with all GRASS
63 void init_map_data(map_data &map)
64 {
65     for (int c = 0; c < MAX_MAP_COLS; c++)
66     {
67         for (int r = 0; r < MAX_MAP_ROWS; r++)
68         {
69             map.tiles[c][r].kind = GRASS_TILE;
70         }
71     }
72 }
73
74 void init_explorer_data(explorer_data &data)
75 {
76     init_map_data(data.map);
77 }
78
79 /**
80  * Returns the color for the indicated tile kind.
81  *
82  * @param kind the tile kind to get the color for
83  * @return color the color for the tile kind
84  */
85 color color_for_tile_kind(tile_kind kind)
86 {
87     switch (kind)
88     {
89     case WATER_TILE:
90         return COLOR_BLUE;
91     case GRASS_TILE:
92         return COLOR_LAWN_GREEN;
93     case SAND_TILE:
94         return COLOR_BLANCHED_ALMOND;
95     case DIRT_TILE:
96         return COLOR_ROSY_BROWN;
97     default:
98         return COLOR_WHITE;
99     }
100 }
101
102 /**
103  * Draw a tile at the indicated location.
104  *
105  * @param tile the tile to draw, passed by constant reference
106  * @param x the x location of the top left of the tile
107  * @param y the y location of the top left of the tile
108  */
109 void draw_tile(const tile_data &tile, double x, double y)
110 {
111     fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
112 }
113
114 // draw the map with position
115 void draw_map(const map_data &map)
116 {
117     for (int c = 0; c < MAX_MAP_COLS; c++)
118     {
119         for (int r = 0; r < MAX_MAP_ROWS; r++)
120         {
121             draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
122         }
123     }
124 }
125
126 // random the map
127 void random_map(map_data &map)
128 {
129     for (int c = 0; c < MAX_MAP_COLS; c++)
130     {
131         for (int r = 0; r < MAX_MAP_ROWS; r++)
132         {
133             map.tiles[c][r].kind = (tile_kind)rnd(NUM_OF_TILE_KIND);
134         }
135     }
136 }
137
138 /**
139  * Shift + E to enter edit mode, ESC to exit
140  *
141  * In edit mode, type R to random map
142  */
143 void handle_input(explorer_data &data)
144 {
145     if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
146     {
147         data.state = EDIT_STATE;
148     }
149     else if (key_typed(ESC_KEY))
150     {
151         data.state = PLAY_STATE;
152     }
153     else if (data.state == EDIT_STATE)
154     {
155         if (key_typed(R_KEY))
156         {
157             random_map(data.map);
158         }
159     }
160 }

```



**Reflect:**

- What was the most interesting thing you learnt through this task, and why?

> I learned about how to build a complex and huge project step by step. First need to build a frame about this project, separate a big problem to many small problem.

- How are arrays and loops connected?

> Arrays in c++ is very inconvenient to do some operation. Using loop we can do some operation like reading, adding, removing or use it. Every time using loop in array, need to using int *i* to looping arrays[*i*].

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I use 32 hours for this task. I many time to doing map explorer and rock dodge. When I finish the initial version of rock dodge using circle, I want to changing it to sprite, I face many problem so then I use bitmap to replace sprite but it also cannot work, the position of rock and player get wrong and I cannot fix it. I debug very long time to fix this.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude to guide me doing the map explorer to add some function.
In the Rock dodge game, I ask about how to remove the marked rock from the array and also help me to check the position problem.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:
- Engage with feedback and discussion to get this signed off.